UNIVERSIDAD CASA GRANDE

Maestría · Taller *No & Low Code Tools vs AI Assisted Coding*

S1-Notebook: Modelo ARIMA para predicción de ventas mensuales

Estudiante: Kevin Barrera

Docente: Mtr. Raquel Vargas

Fecha: 21 de octubre de 2026

Repositorio: https://github.com/Alexis-VsCode/sales-forecasting-demo

Predicción de ventas mensuales utilizando un modelo SARIMA

Taller: *No & Low Code Tools vs AI Assisted Coding* (Maestría, UCG Casa Grande), Mtr. Raquel Vargas. **Actividad:** S1-Notebook: Modelo ARIMA para predicción de ventas mensuales. **Estudiante:** Kevin Barrera

Objetivo. Desarrollar, con asistencia de IA, un modelo ARIMA estacional (SARIMA) que pronostique la serie de ventas mensuales, validarlo con una partición temporal entrenamiento/prueba y compararlo con pronósticos de referencia (*baselines*).

Dataset. `src/data/stores_sales_forecasting.csv` (Superstore, categoría *Furniture*): 2121 transacciones con fecha de pedido (`Order Date`) entre el 2014-01-06 y el 2017-12-30. Las ventas (`Sales`) se suman por mes, lo que da una serie de 48 observaciones (2014-01 a 2017-12). El archivo CSV original no se modifica.

Uso de asistencia de IA. El código de la sesión en vivo se siguió con GitHub Copilot Chat en Visual Studio Code, y esta versión final se completó y revisó con apoyo de un asistente de código de IA. Lo que proponía la IA se contrastó con las salidas reales del código. Por ejemplo, se corrigieron dos defectos del template: la agregación semanal (`freq='W'`), cuando la consigna pide una serie mensual, y la ruta de datos `../../src/data/` , que no existe desde la carpeta `notebooks/` .

Reproducibilidad. Todas las tablas, cifras y gráficos se generan al ejecutar el notebook de principio a fin (*Restart & Run All*). Los comentarios de interpretación se redactaron a partir de esas salidas. Las funciones auxiliares están en el paquete `src/sales_forecasting/` y en `src/modeling/arima_trainer.py` .

In [1]:
```
import itertools
import sys
import warnings
from pathlib import Path

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from statsmodels.tsa.stattools import adfuller
from statsmodels.tsa.seasonal import seasonal_decompose
from statsmodels.stats.diagnostic import acorr_ljungbox
from statsmodels.tsa.stattools import acf
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.graphics.gofplots import qqplot
from statsmodels.tsa.arima.model import ARIMA

# Silenciamos avisos de statsmodels (convergencia, frecuencia) como en el
template
warnings.filterwarnings("ignore", category=UserWarning,
module="statsmodels")
warnings.filterwarnings("ignore", category=FutureWarning)

np.random.seed(42)  # ARIMA es determinista; la semilla se fija por
```

```
reproducibilidad general
plt.rcParams.update({
    "figure.dpi": 100,
    "axes.grid": True,
    "grid.alpha": 0.3,
    "axes.spines.top": False,
    "axes.spines.right": False,
})
COLORES = {"historico": "#2a78d6", "sarima": "#eb6834", "snaive":
"#1baf7a", "real": "#0b0b0b"}
MESES = ["Ene", "Feb", "Mar", "Abr", "May", "Jun", "Jul", "Ago", "Sep",
"Oct", "Nov", "Dic"]
```

Para no depender del directorio desde el que se abre el notebook, se busca hacia arriba la carpeta que contiene `pyproject.toml` (la raíz del repositorio) y se agrega `<raíz>/src` a `sys.path`. Así se pueden importar `sales_forecasting` y `modeling.arima_trainer`.

In [2]:
```
def encontrar_raiz_repo(inicio: Path) -> Path:
    """Sube por los directorios hasta encontrar el que contiene
pyproject.toml."""
    for carpeta in (inicio, *inicio.parents):
        if (carpeta / "pyproject.toml").is_file():
            return carpeta
    raise FileNotFoundError("No se encontró la raíz del repositorio
(pyproject.toml)")


RAIZ = encontrar_raiz_repo(Path.cwd().resolve())
if str(RAIZ / "src") not in sys.path:
    sys.path.insert(0, str(RAIZ / "src"))

from modeling.arima_trainer import select_arima_order  # contrato original
del taller
from sales_forecasting.arima import forecast, grid_search_arima
from sales_forecasting.baselines import (
    moving_average_forecast,
    naive_forecast,
    seasonal_naive_forecast,
)
from sales_forecasting.metrics import evaluate_forecast

print("Raíz del repositorio encontrada:", RAIZ.name)
```

```
Raíz del repositorio encontrada: sales-forecasting-demo
```

Exploración de los datos

In [3]:
```
# El template usaba "../../src/data/", que no existe desde notebooks/; la
ruta correcta es ../src/data/
datapath = RAIZ / "src" / "data"
print("Carpeta de datos:", datapath.relative_to(RAIZ))
```

```
Carpeta de datos: src/data
```

In [4]:
```
dataset = pd.read_csv(
    datapath / "stores_sales_forecasting.csv",
    encoding="latin-1",
```

```
    encoding_errors="replace"
)
print(f"Filas: {dataset.shape[0]}, columnas: {dataset.shape[1]}")
dataset.head().T  # traspuesto para que las 21 columnas sean legibles
```

```
Filas: 2121, columnas: 21
```

Out[4]:

	0	1	2	3	4
Row ID	1	2	4	6	11
Order ID	CA-2016-152156	CA-2016-152156	US-2015-108966	CA-2014-115812	CA-2014-115812
Order Date	11/8/2016	11/8/2016	10/11/2015	6/9/2014	6/9/2014
Ship Date	11/11/2016	11/11/2016	10/18/2015	6/14/2014	6/14/2014
Ship Mode	Second Class	Second Class	Standard Class	Standard Class	Standard Class
Customer ID	CG-12520	CG-12520	SO-20335	BH-11710	BH-11710
Customer Name	Claire Gute	Claire Gute	Sean O'Donnell	Brosina Hoffman	Brosina Hoffman
Segment	Consumer	Consumer	Consumer	Consumer	Consumer
Country	United States	United States	United States	United States	United States
City	Henderson	Henderson	Fort Lauderdale	Los Angeles	Los Angeles
State	Kentucky	Kentucky	Florida	California	California
Postal Code	42420	42420	33311	90032	90032
Region	South	South	South	West	West
Product ID	FUR-BO-10001798	FUR-CH-10000454	FUR-TA-10000577	FUR-FU-10001487	FUR-TA-10001539
Category	Furniture	Furniture	Furniture	Furniture	Furniture
Sub-Category	Bookcases	Chairs	Tables	Furnishings	Tables
Product Name	Bush Somerset Collection Bookcase	Hon Deluxe Fabric Upholstered Stacking Chairs,...	Bretford CR4500 Series Slim Rectangular Table	Eldon Expressions Wood and Plastic Desk Access...	Chromcraft Rectangular Conference Tables
Sales	261.96	731.94	957.5775	48.86	1706.184
Quantity	2	3	5	7	9
Discount	0.0	0.0	0.45	0.0	0.2
Profit	41.9136	219.582	-383.031	14.1694	85.3092

In [5]:

```
dataset.describe().T.round(2)
```

Out[5]:

	count	mean	std	min	25%	50%	75%	max
Row ID	2121.0	5041.64	2885.74	1.00	2568.00	5145.00	7534.00	9991.00
Postal Code	2121.0	55726.56	32261.89	1040.00	22801.00	60505.00	90032.00	99301.00
Sales	2121.0	349.83	503.18	1.89	47.04	182.22	435.17	4416.17
Quantity	2121.0	3.79	2.25	1.00	2.00	3.00	5.00	14.00
Discount	2121.0	0.17	0.18	0.00	0.00	0.20	0.30	0.70
Profit	2121.0	8.70	136.05	-1862.31	-12.85	7.77	33.73	1013.13

In [6]:

```
dataset.describe(include="str").T  # columnas de texto (en pandas 3 el dtype es 'str')
```

Out[6]:

	count	unique	top	freq
Order ID	2121	1764	CA-2016-157749	4
Order Date	2121	889	9/5/2016	10
Ship Date	2121	960	12/16/2015	10
Ship Mode	2121	4	Standard Class	1248
Customer ID	2121	707	SV-20365	15
Customer Name	2121	707	Seth Vernon	15
Segment	2121	3	Consumer	1113
Country	2121	1	United States	2121
City	2121	371	New York City	192
State	2121	48	California	444
Region	2121	4	West	707
Product ID	2121	375	FUR-FU-10004270	16
Category	2121	1	Furniture	2121
Sub-Category	2121	4	Furnishings	957
Product Name	2121	380	KI Adjustable-Height Table	18

In [7]:
```
dataset.dtypes
```

Out[7]:
```
Row ID             int64
Order ID             str
Order Date           str
Ship Date            str
Ship Mode            str
Customer ID          str
Customer Name        str
Segment              str
Country              str
City                 str
State                str
Postal Code        int64
Region               str
Product ID           str
Category             str
Sub-Category         str
Product Name         str
Sales            float64
Quantity           int64
Discount         float64
Profit           float64
dtype: object
```

In [8]:
```
dataset['Order Date'] = pd.to_datetime(dataset['Order Date'],
format="%m/%d/%Y")
```

In [9]:
```
print("Fecha mínima:", dataset['Order Date'].min().date())
print("Fecha máxima:", dataset['Order Date'].max().date())
print("Días distintos con pedidos:", dataset['Order Date'].nunique())
print("Categorías presentes:",
dataset['Category'].value_counts().to_dict())
```

```
Fecha mínima: 2014-01-06
Fecha máxima: 2017-12-30
Días distintos con pedidos: 889
Categorías presentes: {'Furniture': 2121}
```

Agregación mensual

El template agrupaba las ventas por semana (`freq='W'`), pero la consigna pide pronosticar **ventas mensuales**. Por eso se agrega con `freq='MS'` (inicio de mes), lo que deja un índice mensual regular, necesario para el componente estacional de periodo 12.

In [10]:
```
agg_sales = dataset.groupby(pd.Grouper(key='Order Date', freq='MS'))
['Sales'].sum()
agg_sales = agg_sales.asfreq('MS')  # índice mensual regular; NaN si
faltara algún mes
assert agg_sales.notna().all(), "Hay meses sin ventas registradas"
```

In [11]:
```
agg_sales.shape
```

Out[11]:
```
(48,)
```

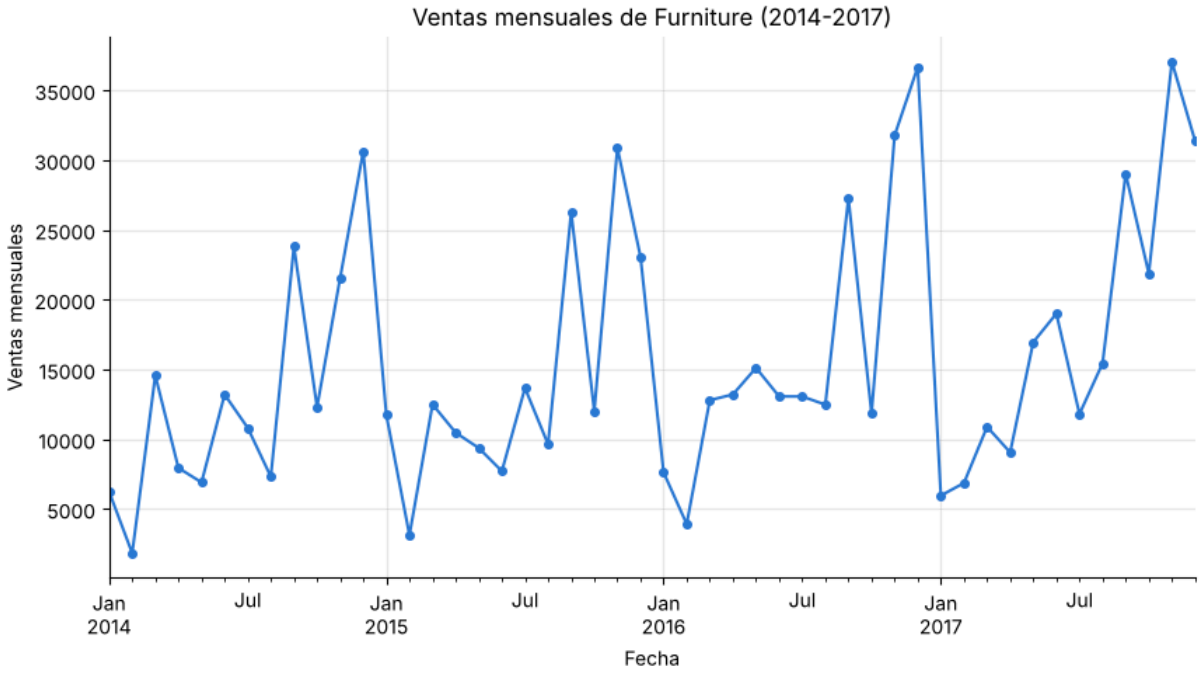
In [12]:
```
agg_sales.describe()
```

Out[12]:
```
count       48.000000
mean     15458.329069
std       9103.800668
min       1839.658000
25%       9297.702400
50%      12650.437500
75%      21644.671575
max      37056.715000
Name: Sales, dtype: float64
```

In [13]:
```
agg_sales.head()
```

Out[13]:
```
Order Date
2014-01-01     6242.525
2014-02-01     1839.658
2014-03-01    14573.956
2014-04-01     7944.837
2014-05-01     6912.787
Freq: MS, Name: Sales, dtype: float64
```

In [14]:
```
ax = agg_sales.plot(figsize=(10, 5), xlabel='Fecha', ylabel="Ventas
mensuales",
                    color=COLORES["historico"], marker="o", markersize=4)
ax.set_title("Ventas mensuales de Furniture (2014-2017)")
plt.show()
```



In [15]:
```
ventas_anuales =
agg_sales.groupby(agg_sales.index.year).sum().rename("Ventas anuales")
ventas_anuales.index.name = "Año"
ventas_anuales.round(2)
```

Out[15]:
```
Año
2014    157192.85
2015    170518.24
2016    198901.44
2017    215387.27
Name: Ventas anuales, dtype: float64
```

Interpretación. La serie mensual tiene 48 observaciones, con una media de 15458.33 y una desviación estándar de 9103.80. Se ve un patrón anual repetido: picos en septiembre, noviembre y diciembre, y valles en enero y febrero (el mínimo es 1839.66, en febrero de 2014). Las ventas anuales crecen todos los años (157192.85 en 2014 y 215387.27 en 2017), lo que sugiere una tendencia creciente moderada además de la estacionalidad.

Test de Dickey-Fuller para verificar estacionariedad

Hipótesis nula (H0): la serie tiene raíz unitaria, es decir, no es estacionaria. Con $\alpha = 0.05$, un p-valor menor que 0.05 lleva a rechazar H0. Se aplica a la serie en niveles y a su diferencia estacional ($y_t - y_{t-12}$).

In [16]:
```
# Sigue el código en vivo
def prueba_adf(serie: pd.Series, nombre: str, alpha: float = 0.05) ->
dict:
    estadistico, p_valor, rezagos, n_obs, criticos, _ =
adfuller(serie.dropna(), autolag="AIC")
    return {
        "serie": nombre,
        "estadístico ADF": round(estadistico, 3),
        "p-valor": f"{p_valor:.2e}",
```

```
        "rezagos usados": rezagos,
        "n obs": n_obs,
        "valor crítico 5%": round(criticos["5%"], 3),
        "¿estacionaria (α=0.05)?": "sí" if p_valor < alpha else "no",
    }


tabla_adf = pd.DataFrame([
    prueba_adf(agg_sales, "Ventas mensuales (niveles)"),
    prueba_adf(agg_sales.diff(12), "Diferencia estacional (s=12)"),
]).set_index("serie")
tabla_adf
```

Out[16]:

serie	estadístico ADF	p-valor	rezagos usados	n obs	valor crítico 5%	¿estacionaria (α=0.05)?
Ventas mensuales (niveles)	-4.699	8.45e-05	0	47	-2.925	sí
Diferencia estacional (s=12)	-6.373	2.32e-08	0	35	-2.949	sí

Interpretación. En niveles, el estadístico ADF (−4.699) está por debajo del valor crítico al 5 % (−2.925) y el p-valor es 8.45e-05, así que se rechaza H0. Con la diferencia estacional, H0 también se rechaza (p-valor 2.32e-08). Hay que leer esto con cautela por tres razones:

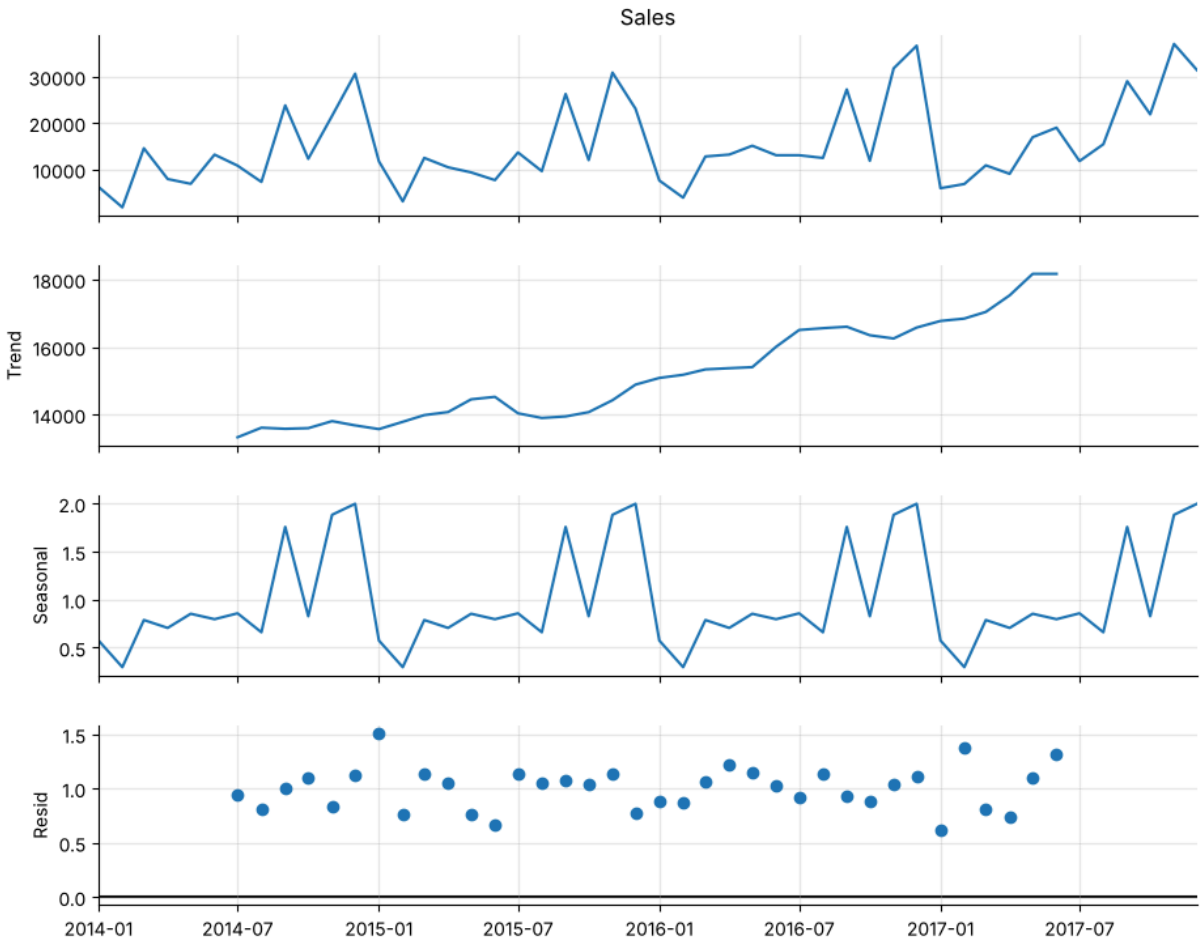
1. El test ADF evalúa una raíz unitaria *regular*, no una *estacional*. La estacionalidad anual es evidente en el gráfico y en la ACF (ver más abajo), y por eso se usa la diferencia estacional D = 1 con s = 12.
2. Con 48 observaciones, el test tiene poca potencia.
3. La estructura de la sesión en vivo, SARIMA(p, 1, q)x(0, 1, 0, 12), aplica además una diferencia regular d = 1. El ADF no la exige, de modo que existe riesgo de **sobrediferenciar**. Esta limitación se retoma en las conclusiones.

Descomposición de la serie de tiempo

In [17]:

```
ventas_descomp = seasonal_decompose(agg_sales, model='multiplicative',
period=12)

fig = ventas_descomp.plot()
fig.set_size_inches(10, 8)
plt.show()
```



In [18]:

```
indices_estacionales = pd.Series(
    ventas_descomp.seasonal.iloc[:12].to_numpy(), index=MESES,
name="Índice estacional"
)
tendencia = ventas_descomp.trend.dropna()
print(f"Tendencia estimada: {tendencia.iloc[0]:.2f}
({tendencia.index[0]:%Y-%m}) -> "
      f"{tendencia.iloc[-1]:.2f} ({tendencia.index[-1]:%Y-%m})")
residuo_descomp = ventas_descomp.resid.dropna()
print(f"Componente residual: mínimo {residuo_descomp.min():.3f}
({residuo_descomp.idxmin():%Y-%m}), "
      f"máximo {residuo_descomp.max():.3f} ({residuo_descomp.idxmax():%Y-
%m})")
indices_estacionales.round(3).to_frame().T
```

```
Tendencia estimada: 13328.46 (2014-07) -> 18168.57 (2017-06)
Componente residual: mínimo 0.619 (2017-01), máximo 1.506 (2015-01)
```

Out[18]:

	Ene	Feb	Mar	Abr	May	Jun	Jul	Ago	Sep	Oct	Nov	Dic
Índice estacional	0.574	0.297	0.788	0.706	0.853	0.797	0.858	0.661	1.757	0.827	1.883	1.998

Interpretación. En la descomposición multiplicativa, los índices estacionales indican cuánto se aleja cada mes del nivel de tendencia. Diciembre (1.998), noviembre (1.883) y septiembre (1.757) quedan entre un 76 % y un 100 % por encima de la tendencia, mientras que febrero (0.297) y enero (0.574) quedan muy por debajo. La tendencia sube de 13328.46 (2014-07) a 18168.57 (2017-06). El componente residual va de 0.619 (enero de 2017) a 1.506 (enero de 2015): hay meses atípicos que la estacionalidad fija no explica, algo esperable con solo cuatro años de datos.

Test de autocorrelación

Test de Ljung-Box sobre la serie de ventas. H0: no hay autocorrelación hasta el rezago indicado. También se muestran algunos valores de la función de autocorrelación (ACF).

In [19]:
```
# Sigue el código en vivo
lb_serie = acorr_ljungbox(agg_sales, lags=[6, 12, 24], return_df=True)
display(lb_serie.round(4))

valores_acf = acf(agg_sales, nlags=24)
pd.Series(valores_acf[[1, 6, 12, 24]], index=["rezago 1", "rezago 6",
"rezago 12", "rezago 24"],
          name="ACF").round(3).to_frame().T
```

	lb_stat	lb_pvalue
6	8.6110	0.1967
12	42.8544	0.0000
24	70.5803	0.0000

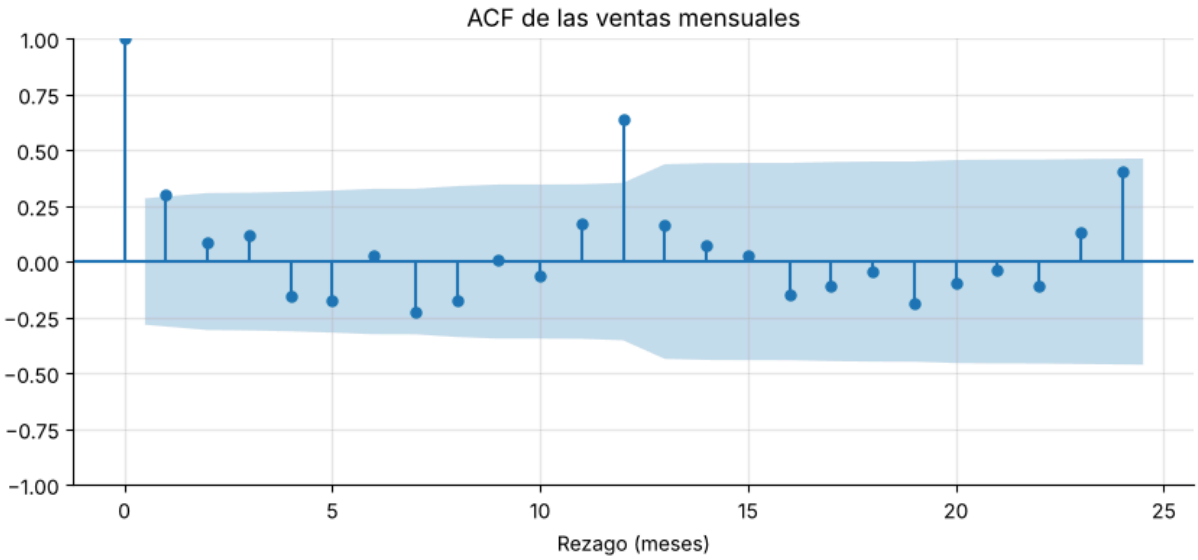
Out[19]:

	rezago 1	rezago 6	rezago 12	rezago 24
ACF	0.297	0.025	0.64	0.4

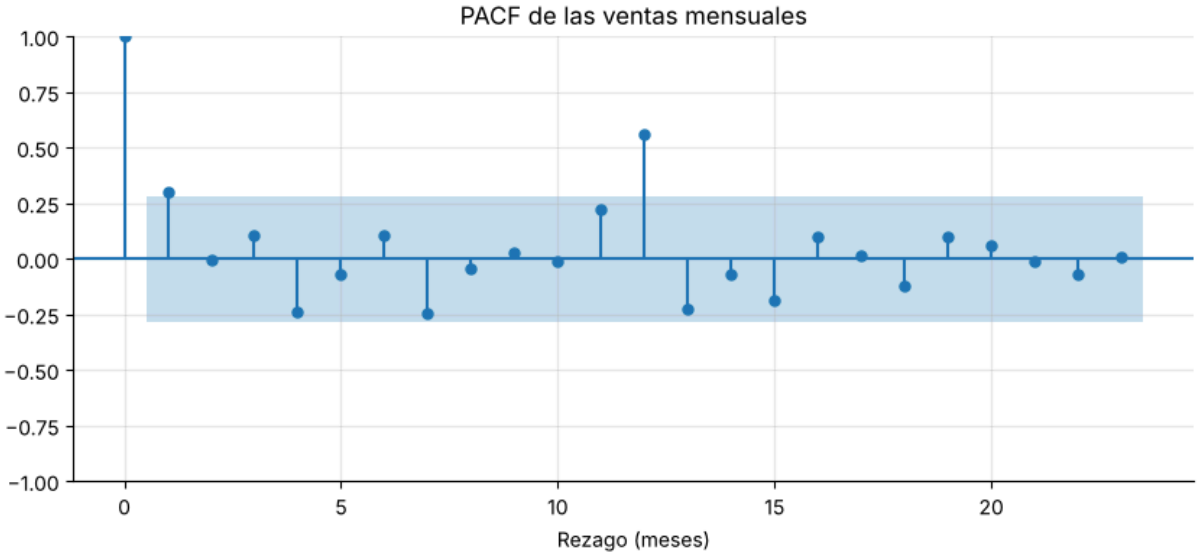
Interpretación. Hasta el rezago 6 no se rechaza H0 (p = 0.1967). Al incluir el rezago 12 el p-valor cae a 0.0000 (redondeado a 4 decimales), y sigue en 0.0000 con 24 rezagos: la serie está autocorrelacionada. La autocorrelación se concentra en el rezago estacional: la ACF vale 0.640 en el rezago 12 y 0.400 en el rezago 24, frente a 0.297 en el rezago 1. Es la firma de una estacionalidad anual que el modelo debe capturar.

Función de autocorrelación (ACF) y autocorrelación parcial (PACF)

In [20]:
```
# Sigue el código en vivo
fig, ax = plt.subplots(figsize=(10, 4))
plot_acf(agg_sales, lags=24, ax=ax, title="ACF de las ventas mensuales")
ax.set_xlabel("Rezago (meses)")
plt.show()
```



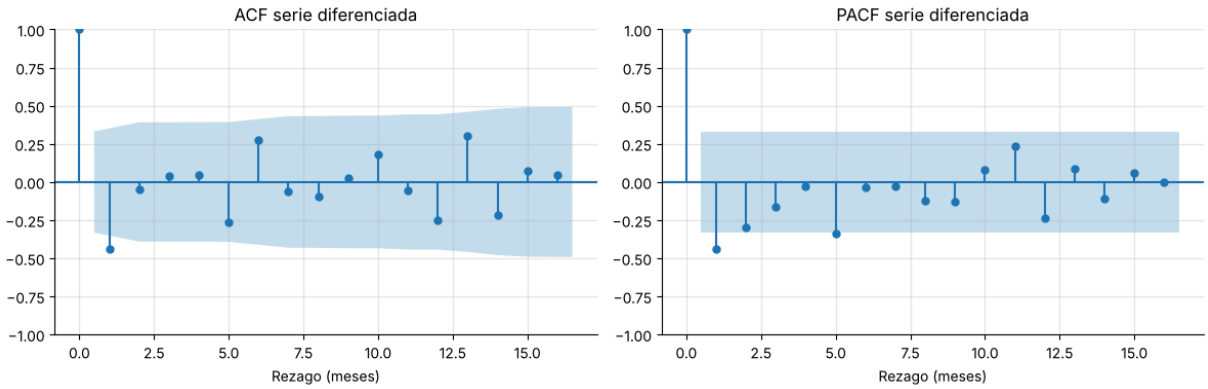
In [21]:
```
# Sigue el código en vivo
# Con 48 observaciones, la PACF admite como máximo 23 rezagos (< n/2)
fig, ax = plt.subplots(figsize=(10, 4))
plot_pacf(agg_sales, lags=23, method="ywm", ax=ax, title="PACF de las
ventas mensuales")
ax.set_xlabel("Rezago (meses)")
plt.show()
```



La ACF y la PACF de la serie con las dos diferencias del modelo (d = 1 y D = 1, s = 12) muestran la estructura que quedaría por modelar con los términos AR/MA.

In [22]:
```
ventas_dif = agg_sales.diff().diff(12).dropna()
print(f"Observaciones tras diferenciar (d=1, D=1, s=12):
{len(ventas_dif)}")
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
plot_acf(ventas_dif, lags=16, ax=axes[0], title="ACF serie diferenciada")
plot_pacf(ventas_dif, lags=16, method="ywm", ax=axes[1], title="PACF serie
diferenciada")
for eje in axes:
    eje.set_xlabel("Rezago (meses)")
plt.tight_layout()
plt.show()
```

```
Observaciones tras diferenciar (d=1, D=1, s=12): 35
```



Interpretación. En la serie original, tanto la ACF como la PACF tienen un pico claro en el rezago 12 que sale de la banda de confianza, mientras que el rezago 1 queda justo en el borde. Esto confirma la estacionalidad anual y justifica D = 1 con s = 12. En la serie doblemente diferenciada (35 observaciones), la ACF y la PACF solo salen con claridad de la banda en el rezago 1 (con valor negativo); en la PACF, el rezago 5 queda en el borde y los rezagos estacionales quedan dentro de la banda. Eso sugiere, como mucho, un término AR o MA de orden bajo. Por eso se buscan p y q entre 0 y 2, y en la extensión se prueban además P y Q.

Modelo ARIMA

Partición temporal: los últimos 12 meses (2017) se reservan como prueba y no se usan para elegir el modelo. Los 36 meses anteriores (2014-2016) forman el entrenamiento.

In [23]:
```
meses = 12
agg_sales_train = agg_sales.iloc[0:len(agg_sales) - meses]
agg_sales_test = agg_sales.tail(meses)
print(f"Total de observaciones para entrenamiento:
{len(agg_sales_train)}")
print(f"Total de observaciones para prueba: {len(agg_sales_test)}")
print(f"Entrenamiento: {agg_sales_train.index.min():%Y-%m} a
{agg_sales_train.index.max():%Y-%m}")
print(f"Prueba:        {agg_sales_test.index.min():%Y-%m} a
{agg_sales_test.index.max():%Y-%m}")
```

```
Total de observaciones para entrenamiento: 36
Total de observaciones para prueba: 12
Entrenamiento: 2014-01 a 2016-12
Prueba:        2017-01 a 2017-12
```

In [24]:
```
#SARIMA=(p,d,q)x(P,D,Q,S)
#Nuestro modelo puede tener la forma: ARIMA=(p, 1, q)x(0, 1, 0, 12)
import warnings
# Suprime las warnings
warnings.filterwarnings("ignore", category=UserWarning,
module="statsmodels")
warnings.filterwarnings("ignore", category=FutureWarning)

# Completamos el código en vivo
ORDEN_ESTACIONAL = (0, 1, 0, 12)

# Búsqueda en rejilla de p, q en {0, 1, 2} con d = 1, SOLO con los datos de
entrenamiento
tabla_aic = grid_search_arima(agg_sales_train, p_values=range(3),
```

```
d_values=range(1, 2),
                                q_values=range(3),
seasonal_order=ORDEN_ESTACIONAL)

# Función del taller (src/modeling/arima_trainer.py): devuelve el orden con
menor AIC
mejor_orden, mejor_aic = select_arima_order(agg_sales_train,
p_values=range(3),
                                            d_values=range(1, 2),
q_values=range(3),

seasonal_order=ORDEN_ESTACIONAL)
print(f"Mejor orden por AIC (solo entrenamiento):
SARIMA{mejor_orden}x{ORDEN_ESTACIONAL}, "
      f"AIC = {mejor_aic:.2f}")
tabla_aic.round(2)
```

```
Mejor orden por AIC (solo entrenamiento): SARIMA(0, 1, 0)x(0, 1, 0, 12), AIC = 467.64
```

Out[24]:

	order	aic	bic
0	(0, 1, 0)	467.64	468.77
1	(1, 1, 1)	468.30	471.71
2	(1, 1, 0)	468.83	471.10
3	(0, 1, 1)	468.95	471.22
4	(2, 1, 1)	470.11	474.65
5	(0, 1, 2)	470.23	473.64
6	(2, 1, 0)	471.45	474.85
7	(2, 1, 2)	471.98	477.66
8	(1, 1, 2)	472.40	476.94

Interpretación. El menor AIC corresponde a SARIMA(0, 1, 0)x(0, 1, 0, 12), con AIC = 467.64. La diferencia con el segundo, (1, 1, 1), es menor que 1 punto de AIC (468.30), así que los dos son prácticamente equivalentes. El criterio favorece al más parsimonioso, que no estima ningún coeficiente AR ni MA.

In [25]:

```
modelo_sarima = ARIMA(agg_sales_train, order=mejor_orden,
seasonal_order=ORDEN_ESTACIONAL).fit()
print(modelo_sarima.summary())
```

```
                                SARIMAX Results
==========================================================================================
Dep. Variable:                             Sales   No. Observations:                   36
Model:             ARIMA(0, 1, 0)x(0, 1, 0, 12)   Log Likelihood                -232.820
Date:                          Thu, 22 Oct 2026   AIC                            467.639
Time:                                  04:12:04   BIC                            468.775
Sample:                              01-01-2014   HQIC                           467.925
                                   - 12-01-2016
Covariance Type:                           opg
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
sigma2      3.418e+07   7.48e+06      4.567      0.000    1.95e+07    4.88e+07
===================================================================================
Ljung-Box (L1) (Q):                   3.63   Jarque-Bera (JB):                 2.89
Prob(Q):                              0.06   Prob(JB):                         0.24
Heteroskedasticity (H):               1.22   Skew:                            -0.54
Prob(H) (two-sided):                  0.79   Kurtosis:                         4.36
===================================================================================

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).
```

El modelo elegido solo estima la varianza del error (`sigma2`). Con d = 1 y D = 1, su pronóstico es $\hat{y}_{T+h} = y_{T+h-12} + (y_T - y_{T-12})$: repite el valor del mismo mes del año anterior (como el *seasonal naive*) y le suma el último cambio interanual observado. Esta propiedad se comprueba numéricamente en la sección siguiente.

Métricas de validación

El pronóstico del SARIMA para los 12 meses de prueba se compara con tres *baselines*:

- **Naive:** repite el último valor observado.
- **Seasonal naive:** repite el mismo mes del año anterior.
- **Media móvil (3):** promedio de los últimos 3 meses.

Métricas:

- **MAE:** error absoluto medio.
- **RMSE:** raíz del error cuadrático medio; penaliza más los errores grandes.
- **sMAPE (%):** error porcentual absoluto simétrico.
- **WAPE (%):** suma de errores absolutos sobre suma de ventas reales.

En las cuatro métricas, un valor menor es mejor.

In [26]:
```python
# Completamos el código en vivo
horizonte = len(agg_sales_test)
nombre_sarima = f"SARIMA{mejor_orden}x{ORDEN_ESTACIONAL}"
pron_sarima = forecast(modelo_sarima, horizonte, alpha=0.05)  # pronóstico
+ IC 95 %

predicciones = pd.DataFrame({
    "Real": agg_sales_test,
    nombre_sarima: pron_sarima["forecast"],
    "Naive": naive_forecast(agg_sales_train, horizonte),
    "Seasonal naive": seasonal_naive_forecast(agg_sales_train, horizonte,
season_length=12),
```

```
    "Media móvil (3)": moving_average_forecast(agg_sales_train, horizonte,
window=3),
})
assert predicciones.notna().all().all(), "Índices de pronóstico
desalineados"

modelos = [col for col in predicciones.columns if col != "Real"]
tabla_metricas = pd.DataFrame(
    {m: evaluate_forecast(predicciones["Real"], predicciones[m]) for m in
modelos}
).T.sort_values("RMSE")
tabla_metricas.index.name = "modelo"
tabla_metricas.round(2)
```

Out[26]:

modelo	MAE	RMSE	sMAPE	WAPE
Seasonal naive	3747.03	4473.38	25.70	20.88
SARIMA(0, 1, 0)x(0, 1, 0, 12)	12219.08	12939.46	59.20	68.08
Media móvil (3)	11688.95	13105.84	59.59	65.12
Naive	18792.78	21085.77	76.79	104.70

In [27]:

```
# Pronóstico mensual del periodo de prueba (con intervalo de confianza del
95 % del SARIMA)
tabla_prueba = predicciones[["Real", nombre_sarima, "Seasonal
naive"]].assign(
    **{"IC 95% inferior": pron_sarima["lower"], "IC 95% superior":
pron_sarima["upper"]}
)
tabla_prueba.index = tabla_prueba.index.strftime("%Y-%m")
display(tabla_prueba.round(2))

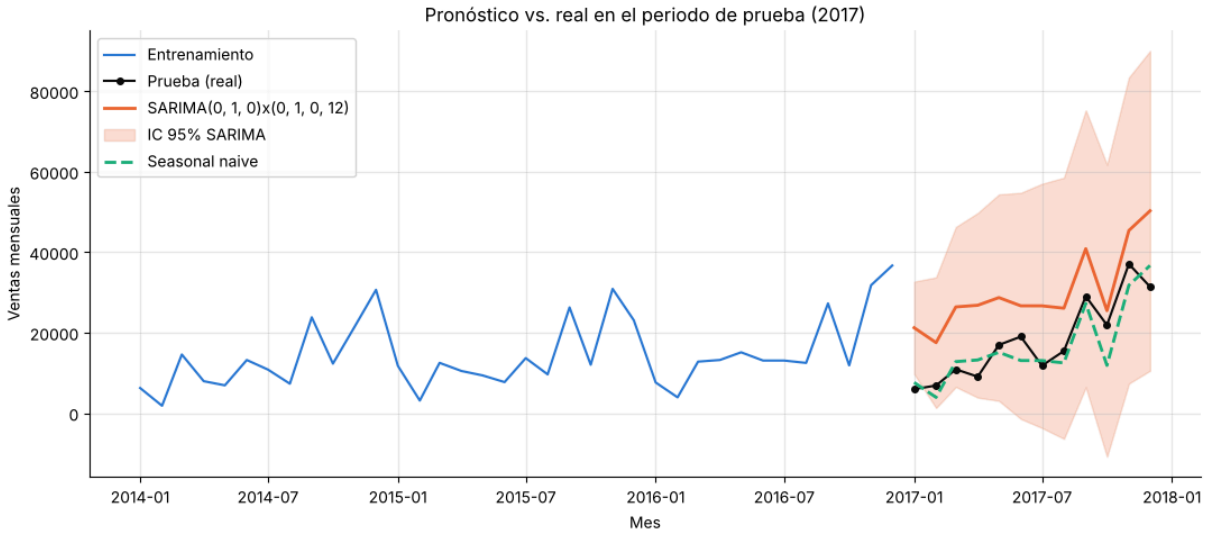
# Comprobación: el SARIMA(0,1,0)x(0,1,0,12) = seasonal naive + último
cambio interanual
desfase = predicciones[nombre_sarima] - predicciones["Seasonal naive"]
cambio_interanual = agg_sales_train.iloc[-1] - agg_sales_train.iloc[-13]
print(f"Diferencia SARIMA - seasonal naive: mín {desfase.min():.2f}, máx
{desfase.max():.2f}")
print(f"Cambio interanual dic-2016 vs dic-2015: {cambio_interanual:.2f}")
real_en_ic = predicciones["Real"].between(pron_sarima["lower"],
pron_sarima["upper"])
print(f"Meses con el valor real dentro del IC 95 %: {real_en_ic.sum()} de
{horizonte} "
      f"(fuera: {', '.join(real_en_ic.index[~real_en_ic].strftime('%Y-
%m'))})")
sobreestima = (predicciones[nombre_sarima] > predicciones["Real"]).sum()
print(f"Meses en que el SARIMA sobreestima la venta real: {sobreestima} de
{horizonte}")
```

	Real	SARIMA(0, 1, 0)x(0, 1, 0, 12)	Seasonal naive	IC 95% inferior	IC 95% superior
2017-01	5964.03	21215.64	7622.74	9757.38	32673.90
2017-02	6866.34	17518.45	3925.55	1314.02	33722.87
2017-03	10893.44	26393.99	12801.09	6547.70	46240.28
2017-04	9065.96	26804.99	13212.09	3888.47	49721.51
2017-05	16957.56	28712.73	15119.84	3091.28	54334.18
2017-06	19008.59	26663.47	13070.57	-1403.42	54730.36
2017-07	11813.02	26661.41	13068.52	-3654.29	56977.12
2017-08	15441.87	26076.13	12483.23	-6332.73	58484.98
2017-09	29028.21	40855.78	27262.88	6481.00	75230.56
2017-10	21884.07	25465.47	11872.58	-10768.73	61699.67
2017-11	37056.72	45376.52	31783.63	7373.77	83379.27
2017-12	31407.47	50271.61	36678.71	10579.03	89964.19

```
Diferencia SARIMA - seasonal naive: mín 13592.90, máx 13592.90
Cambio interanual dic-2016 vs dic-2015: 13592.90
Meses con el valor real dentro del IC 95 %: 11 de 12 (fuera: 2017-01)
Meses en que el SARIMA sobreestima la venta real: 12 de 12
```

In [28]:

```
# Gráfico de forecast
fig, ax = plt.subplots(figsize=(11, 5))
ax.plot(agg_sales_train.index, agg_sales_train,
color=COLORES["historico"], label="Entrenamiento")
ax.plot(agg_sales_test.index, agg_sales_test, color=COLORES["real"],
marker="o", markersize=4,
        label="Prueba (real)")
ax.plot(pron_sarima.index, pron_sarima["forecast"],
color=COLORES["sarima"], linewidth=2,
        label=nombre_sarima)
ax.fill_between(pron_sarima.index, pron_sarima["lower"],
pron_sarima["upper"],
                color=COLORES["sarima"], alpha=0.2, label="IC 95% SARIMA")
ax.plot(predicciones.index, predicciones["Seasonal naive"],
color=COLORES["snaive"],
        linestyle="--", linewidth=2, label="Seasonal naive")
ax.set(title="Pronóstico vs. real en el periodo de prueba (2017)",
xlabel="Mes",
       ylabel="Ventas mensuales")
ax.legend(loc="upper left")
plt.tight_layout()
plt.show()
```



Interpretación. En la prueba (2017) el mejor modelo es el **seasonal naive** (MAE 3747.03, RMSE 4473.38, sMAPE 25.70 %, WAPE 20.88 %). El SARIMA(0, 1, 0)x(0, 1, 0, 12) queda bastante por detrás (MAE 12219.08, RMSE 12939.46, sMAPE 59.20 %, WAPE 68.08 %): su RMSE es casi el triple del *seasonal naive*, mejora apenas a la media móvil en RMSE (13105.84), que incluso tiene un MAE algo menor (11688.95 frente a 12219.08), y solo supera con claridad al naive (RMSE 21085.77).

La causa se ve en la comprobación numérica. El pronóstico del SARIMA es exactamente el *seasonal naive* más 13592.90, el salto entre diciembre de 2016 y diciembre de 2015. Ese salto fue atípico y no se repitió en 2017, de modo que el modelo sobreestima los 12 meses de prueba. El intervalo del 95 % contiene 11 de los 12 valores reales (todos salvo enero de 2017), pero es tan ancho (su límite inferior llega a ser negativo) que refleja una gran incertidumbre, no precisión. **Con esta configuración, el SARIMA de la sesión en vivo no supera al baseline estacional.**

Análisis de residuos

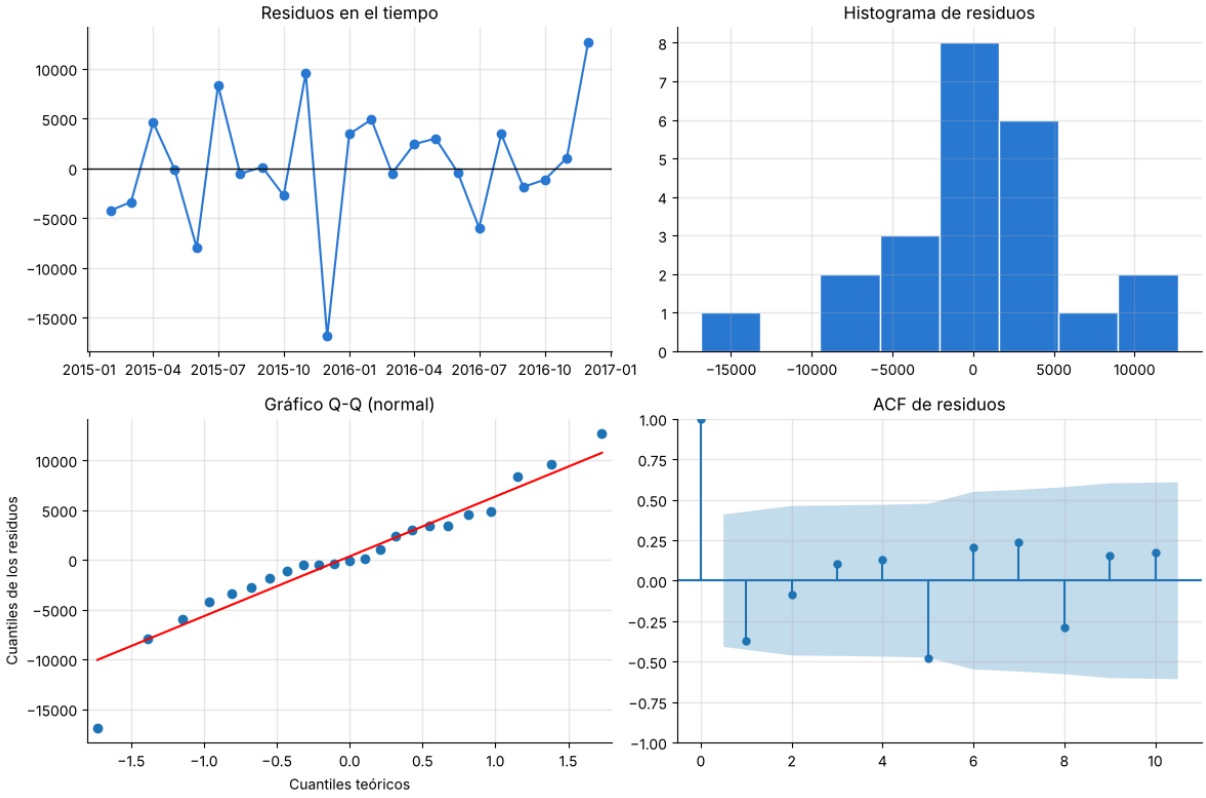
Los primeros 13 residuos (d + D·s = 1 + 12) dependen de las condiciones iniciales de la diferenciación y no son informativos, así que se excluyen (*burn-in*). Quedan 23 residuos del periodo de entrenamiento.

In [29]:
```
# Completamos el código en vivo
BURN_IN = mejor_orden[1] + ORDEN_ESTACIONAL[1] * ORDEN_ESTACIONAL[3]  # 1
+ 1*12 = 13
residuos = modelo_sarima.resid.iloc[BURN_IN:]
print(f"Residuos analizados: {len(residuos)} de {len(modelo_sarima.resid)}
"
      f"(media = {residuos.mean():.2f}, desviación estándar =
{residuos.std():.2f})")

fig, axes = plt.subplots(2, 2, figsize=(12, 8))
axes[0, 0].plot(residuos.index, residuos, marker="o",
color=COLORES["historico"])
axes[0, 0].axhline(0, color=COLORES["real"], linewidth=1)
axes[0, 0].set_title("Residuos en el tiempo")
axes[0, 1].hist(residuos, bins=8, color=COLORES["historico"],
edgecolor="white")
```

```
axes[0, 1].set_title("Histograma de residuos")
qqplot(residuos, line="s", ax=axes[1, 0])
axes[1, 0].set(title="Gráfico Q-Q (normal)", xlabel="Cuantiles teóricos",
               ylabel="Cuantiles de los residuos")
plot_acf(residuos, lags=10, ax=axes[1, 1], title="ACF de residuos")
plt.tight_layout()
plt.show()
```

```
Residuos analizados: 23 de 36 (media = 351.98, desviación estándar = 6144.41)
```



In [30]:
```
# Test de autocorrelación de Ljung-Box de residuos
lb_residuos = acorr_ljungbox(residuos, lags=[6, 12], return_df=True)
lb_residuos.round(4)
```

Out[30]:

	lb_stat	lb_pvalue
6	13.2957	0.0386
12	26.0748	0.0105

Interpretación. El Ljung-Box sobre los residuos útiles da p = 0.0386 (rezago 6) y p = 0.0105 (rezago 12). Ambos son menores que 0.05, así que se rechaza que los residuos sean ruido blanco: **queda autocorrelación sin modelar**. Con solo 23 residuos, el histograma y el gráfico Q-Q permiten una lectura aproximada de la normalidad y no una conclusión firme. El diagnóstico confirma que el modelo de la sesión es incompleto, y por eso se prueba la extensión siguiente.

Extensión (exploratoria): términos estacionales P y Q

Se amplía la búsqueda a SARIMA(p, 1, q)x(P, 1, Q, 12), con p, q ∈ {0, 1, 2} y P, Q ∈ {0, 1}. Se mantienen d = 1 y D = 1 para que los AIC sean comparables, ya que el AIC solo es comparable entre modelos con la misma diferenciación. El orden se elige **solo por**

AIC sobre el entrenamiento. El conjunto de prueba se usa una única vez, después de elegir, para informar el resultado; no se usó para seleccionar.

In [31]:
```
resultados_ext = []
for P, Q in itertools.product([0, 1], [0, 1]):
    estacional = (P, 1, Q, 12)
    tabla = grid_search_arima(agg_sales_train, p_values=range(3),
d_values=range(1, 2),
                              q_values=range(3),
seasonal_order=estacional)
    tabla["seasonal_order"] = [estacional] * len(tabla)
    resultados_ext.append(tabla)

tabla_ext = pd.concat(resultados_ext).sort_values("aic",
kind="stable").reset_index(drop=True)
print(f"Modelos ajustados: {len(tabla_ext)}")
tabla_ext.head(10).round(2)
```

```
Modelos ajustados: 36
```

Out[31]:

	order	aic	bic	seasonal_order
0	(0, 1, 0)	465.94	468.21	(1, 1, 0, 12)
1	(0, 1, 0)	466.15	468.42	(0, 1, 1, 12)
2	(1, 1, 1)	466.90	471.44	(1, 1, 0, 12)
3	(0, 1, 1)	467.04	470.45	(1, 1, 0, 12)
4	(1, 1, 1)	467.08	471.62	(0, 1, 1, 12)
5	(1, 1, 0)	467.18	470.59	(1, 1, 0, 12)
6	(0, 1, 1)	467.23	470.64	(0, 1, 1, 12)
7	(1, 1, 0)	467.34	470.75	(0, 1, 1, 12)
8	(0, 1, 0)	467.64	468.77	(0, 1, 0, 12)
9	(0, 1, 0)	468.09	471.50	(1, 1, 1, 12)

In [32]:
```
orden_ext = tuple(int(v) for v in tabla_ext.loc[0, "order"])
estacional_ext = tuple(int(v) for v in tabla_ext.loc[0, "seasonal_order"])
nombre_ext = f"SARIMA{orden_ext}x{estacional_ext}"
print(f"Elegido por AIC (entrenamiento): {nombre_ext}, AIC =
{tabla_ext.loc[0, 'aic']:.2f}")

modelo_ext = ARIMA(agg_sales_train, order=orden_ext,
seasonal_order=estacional_ext).fit()
print(modelo_ext.summary())
```

```
Elegido por AIC (entrenamiento): SARIMA(0, 1, 0)x(1, 1, 0, 12), AIC = 465.94
                                     SARIMAX Results
==========================================================================================
Dep. Variable:                             Sales   No. Observations:                   36
Model:             ARIMA(0, 1, 0)x(1, 1, 0, 12)   Log Likelihood                -230.971
Date:                          Thu, 22 Oct 2026   AIC                            465.942
Time:                                  04:12:10   BIC                            468.213
Sample:                              01-01-2014   HQIC                           466.513
                                   - 12-01-2016
Covariance Type:                           opg
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
ar.S.L12      -0.2494      0.119     -2.100      0.036      -0.482      -0.017
sigma2      3.079e+07   6.63e-10   4.64e+16      0.000    3.08e+07    3.08e+07
===================================================================================
Ljung-Box (L1) (Q):                   4.29   Jarque-Bera (JB):                 0.81
Prob(Q):                              0.04   Prob(JB):                         0.67
Heteroskedasticity (H):               0.83   Skew:                            -0.31
Prob(H) (two-sided):                  0.80   Kurtosis:                         3.67
===================================================================================

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).
[2] Covariance matrix is singular or near-singular, with condition number 6.54e+32. Standa
rd errors may be unstable.
```

In [33]:

```python
# Evaluación única en el conjunto de prueba
pron_ext = forecast(modelo_ext, horizonte, alpha=0.05)
tabla_metricas_ext = pd.concat([
    tabla_metricas,
    pd.DataFrame({f"{nombre_ext} (extensión)":
evaluate_forecast(agg_sales_test,

pron_ext["forecast"])}).T,
]).sort_values("RMSE")
tabla_metricas_ext.index.name = "modelo"
display(tabla_metricas_ext.round(2))

residuos_ext = modelo_ext.resid.iloc[BURN_IN:]
print("Ljung-Box de residuos del modelo de la extensión:")
acorr_ljungbox(residuos_ext, lags=[6, 12], return_df=True).round(4)
```

modelo	MAE	RMSE	sMAPE	WAPE
Seasonal naive	3747.03	4473.38	25.70	20.88
SARIMA(0, 1, 0)x(1, 1, 0, 12) (extensión)	9743.64	10574.03	51.22	54.29
SARIMA(0, 1, 0)x(0, 1, 0, 12)	12219.08	12939.46	59.20	68.08
Media móvil (3)	11688.95	13105.84	59.59	65.12
Naive	18792.78	21085.77	76.79	104.70

```
Ljung-Box de residuos del modelo de la extensión:
```

Out[33]:

	lb_stat	lb_pvalue
6	9.8175	0.1326
12	14.7127	0.2575

Interpretación (exploratoria). El AIC elige SARIMA(0, 1, 0)x(1, 1, 0, 12), con AIC = 465.94 frente a 467.64 del modelo de la sesión. Agrega un término autorregresivo estacional con coeficiente −0.249 (p = 0.036; statsmodels advierte que la matriz de covarianza es casi singular, así que los errores estándar deben tomarse con cautela),

y sus residuos ya no muestran autocorrelación significativa (Ljung-Box p = 0.1326 en el rezago 6 y p = 0.2575 en el rezago 12).

En la prueba mejora al SARIMA de la sesión (RMSE 10574.03 frente a 12939.46; WAPE 54.29 % frente a 68.08 %), pero sigue **muy por detrás del seasonal naive** (RMSE 4473.38). Las diferencias de AIC son pequeñas, el entrenamiento tiene solo 36 meses y la prueba se evaluó una única vez, así que no conviene sobreinterpretar el resultado. La extensión indica una dirección de mejora, no un modelo definitivo.

Pronóstico de los próximos 12 meses (2018)

Se reentrena el modelo con toda la serie (48 meses) usando el orden elegido en la sesión, SARIMA(0, 1, 0)x(0, 1, 0, 12), y se pronostica 2018 con un intervalo de confianza del 95 %. Como referencia se incluye el *seasonal naive*, que fue el mejor modelo en la prueba (repite las ventas de 2017).

In [34]:
```
modelo_final = ARIMA(agg_sales, order=mejor_orden,
seasonal_order=ORDEN_ESTACIONAL).fit()
pron_2018 = forecast(modelo_final, 12, alpha=0.05).rename(columns={
    "forecast": "Pronóstico SARIMA", "lower": "IC 95% inferior", "upper":
"IC 95% superior"})
pron_2018["Seasonal naive (referencia)"] =
seasonal_naive_forecast(agg_sales, 12, season_length=12)
pron_2018.index.name = "Mes"

totales = pd.Series({
    "Ventas reales 2017": agg_sales.loc["2017"].sum(),
    "Pronóstico SARIMA 2018": pron_2018["Pronóstico SARIMA"].sum(),
    "Seasonal naive 2018": pron_2018["Seasonal naive (referencia)"].sum(),
}, name="Total anual").round(2)
print(totales.to_string())
print(f"Cambio interanual dic-2017 vs dic-2016: {agg_sales.iloc[-1] -
agg_sales.iloc[-13]:.2f}")

tabla_2018 = pron_2018.copy()
tabla_2018.index = tabla_2018.index.strftime("%Y-%m")
tabla_2018.round(2)
```

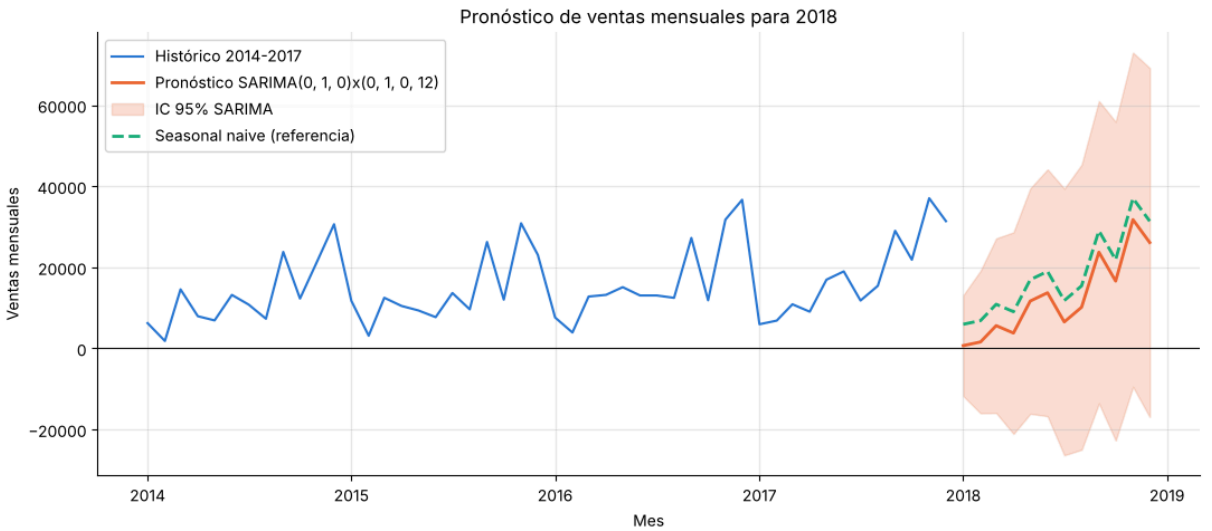
```
Ventas reales 2017        215387.27
Pronóstico SARIMA 2018    152132.29
Seasonal naive 2018       215387.27
Cambio interanual dic-2017 vs dic-2016: -5271.25
```

Out[34]:

Mes	Pronóstico SARIMA	IC 95% inferior	IC 95% superior	Seasonal naive (referencia)
2018-01	692.78	-11732.48	13118.05	5964.03
2018-02	1595.09	-15976.89	19167.06	6866.34
2018-03	5622.20	-15898.99	27143.38	10893.44
2018-04	3794.71	-21055.81	28645.23	9065.96
2018-05	11686.31	-16097.42	39470.04	16957.56
2018-06	13737.34	-16698.21	44172.89	19008.59
2018-07	6541.77	-26332.38	39415.93	11813.02
2018-08	10170.63	-24973.32	45314.57	15441.87
2018-09	23756.96	-13518.83	61032.74	29028.21
2018-10	16612.82	-22679.31	55904.95	21884.07
2018-11	31785.47	-9424.47	72995.40	37056.72
2018-12	26136.22	-16906.15	69178.59	31407.47

In [35]:

```
fig, ax = plt.subplots(figsize=(11, 5))
ax.plot(agg_sales.index, agg_sales, color=COLORES["historico"],
label="Histórico 2014-2017")
ax.plot(pron_2018.index, pron_2018["Pronóstico SARIMA"],
color=COLORES["sarima"], linewidth=2,
        label=f"Pronóstico {nombre_sarima}")
ax.fill_between(pron_2018.index, pron_2018["IC 95% inferior"],
pron_2018["IC 95% superior"],
                color=COLORES["sarima"], alpha=0.2, label="IC 95% SARIMA")
ax.plot(pron_2018.index, pron_2018["Seasonal naive (referencia)"],
color=COLORES["snaive"],
        linestyle="--", linewidth=2, label="Seasonal naive (referencia)")
ax.axhline(0, color=COLORES["real"], linewidth=0.8)
ax.set(title="Pronóstico de ventas mensuales para 2018", xlabel="Mes",
ylabel="Ventas mensuales")
ax.legend(loc="upper left")
plt.tight_layout()
plt.show()
```



Interpretación. El SARIMA reentrenado pronostica un total de 152132.29 para 2018, frente a 215387.27 vendidos en 2017. Eso sería una caída de alrededor del 29 %, que contradice el crecimiento observado todos los años. La razón es la misma propiedad

de antes: el modelo traslada a todos los meses el último cambio interanual (diciembre de 2017 frente a diciembre de 2016, −5271.25), y por eso enero y febrero de 2018 quedan en valores muy bajos (692.78 y 1595.09). Además, los límites inferiores del IC 95 % son negativos en todos los meses, lo que no tiene sentido para ventas. Este pronóstico ilustra el procedimiento, pero **no debería usarse para decisiones** sin mejorar el modelo; con la evidencia de la prueba, el *seasonal naive* (215387.27) es la referencia más defendible.

Conclusiones

1. **Datos y preparación.** Las 2121 transacciones de Furniture se agregaron en una serie **mensual** de 48 observaciones (2014-01 a 2017-12). Se corrigieron dos defectos del template: la frecuencia semanal (`'W'`), cuando la consigna pide mensual, y la ruta de datos incorrecta.
2. **Estructura de la serie.** Hay una estacionalidad anual fuerte: índices estacionales de 1.998 en diciembre y 0.297 en febrero, y ACF de 0.640 en el rezago 12. También hay una tendencia creciente moderada (ventas anuales de 157192.85 en 2014 a 215387.27 en 2017). El ADF rechaza la raíz unitaria tanto en niveles (p = 8.45e-05) como con diferencia estacional.
3. **Modelo de la sesión.** La búsqueda por AIC, solo con los 36 meses de entrenamiento, eligió SARIMA(0, 1, 0)x(0, 1, 0, 12), con AIC = 467.64. En la prueba (2017) obtuvo MAE 12219.08, RMSE 12939.46, sMAPE 59.20 % y WAPE 68.08 %. **No supera al seasonal naive** (MAE 3747.03, RMSE 4473.38, sMAPE 25.70 %, WAPE 20.88 %). Sus residuos conservan autocorrelación (Ljung-Box p = 0.0386 en el rezago 6 y p = 0.0105 en el rezago 12).
4. **Por qué falla.** Con d = 1 y D = 1 y sin términos AR/MA, el pronóstico equivale al seasonal naive más el último cambio interanual (+13592.90 en la prueba, −5271.25 para 2018). Un único salto atípico sesga todo el horizonte.
5. **Extensión.** Al permitir P y Q, el AIC elige SARIMA(0, 1, 0)x(1, 1, 0, 12) (AIC 465.94). Sus residuos ya no muestran autocorrelación significativa y mejora al modelo de la sesión en la prueba (RMSE 10574.03), pero sigue lejos del seasonal naive.
6. **Limitaciones.** La serie es corta (48 observaciones, 36 para entrenar y solo 23 residuos útiles), cubre una sola categoría (Furniture), tiene meses atípicos y no usa variables exógenas (promociones, calendario, precios). La validación consiste en un único corte temporal de 12 meses. Además, el ADF sugiere que la diferencia regular d = 1 podría sobrediferenciar. Como trabajo futuro se proponen: comparar d = 0 con D = 1 usando validación cruzada temporal (*rolling origin*), probar transformaciones logarítmicas y otros modelos (ETS, regresión con estacionalidad).
7. **Asistencia de IA frente a criterio humano.** Los asistentes de código (GitHub Copilot Chat en la sesión y un asistente de IA en la versión final) aceleran la escritura del código. Sin embargo, el template de partida tenía decisiones incorrectas para la consigna (agregación semanal y ruta de datos rota), y un asistente puede reproducirlas si no se le da el contexto. El criterio humano fue necesario para:

- verificar que la frecuencia correspondiera al enunciado;
- elegir el modelo solo con el entrenamiento, para evitar fuga de información;
- comparar contra baselines;
- revisar los residuos;
- informar con honestidad que el modelo no supera a la referencia.